

SMARTHAND AI

Protótipo de Mão Robótica Inteligente Documentação Técnica de Hardware, Software e Impressão 3D



Versão de documentação: 1.0 | Agosto de 2026

1. Controle do documento e aviso de escopo

Campo	Definição
Projeto	SmartHand AI
Tipo	Projeto acadêmico / protótipo experimental
Escopo desta versão	Mão robótica impressa em 3D, acionada por tendões e servomotores, com MVP por visão computacional
Evolução prevista	Controle por EMG, feedback de força e classificação de gestos
Status	Conceito técnico + código-base para prototipagem

Aviso importante: A imagem preta usada como inspiração é um render conceitual, não um arquivo CAD dimensionado e não um STL pronto para impressão. A geometria final deve ser modelada e validada em CAD, com folgas, limites de curso, resistência mecânica e alojamentos ajustados aos componentes físicos medidos.

Uso experimental: O projeto deve ser tratado como plataforma educacional de robótica e tecnologia assistiva. Não deve ser apresentado nem utilizado como prótese clínica, diagnóstica ou terapêutica.

2. Resumo executivo

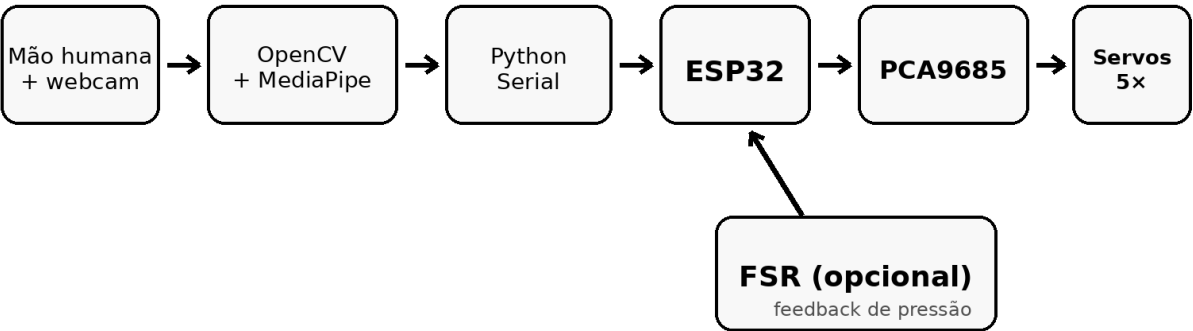
O SmartHand AI é uma proposta de mão robótica/protética experimental com estrutura impressa em 3D, articulações segmentadas e acionamento dos dedos por cabos semelhantes a tendões. Os atuadores ficam concentrados no punho ou pequeno módulo de antebraço, liberando espaço nas falanges e permitindo uma aparência externa mais limpa.

O primeiro MVP usa uma webcam e visão computacional para capturar os movimentos de uma mão de referência. OpenCV realiza a captura de vídeo, MediaPipe Hand Landmarker identifica os pontos da mão, Python converte a flexão dos dedos em comandos e um ESP32 transmite esses comandos ao controlador PWM PCA9685. O PCA9685 aciona cinco servomotores, um por dedo.

A evolução do projeto substitui ou complementa a webcam com sensores EMG para detectar atividade muscular e adiciona sensores FSR nas pontas dos dedos para limitar a pressão aplicada aos objetos.

Arquitetura SmartHand AI - MVP

Visão computacional para validar mecânica e controle dos cinco dedos



Evolução futura: EMG + classificação de gestos personalizados.

Figura 1 - Arquitetura funcional do MVP SmartHand AI
Fonte/crédito: Diagrama produzido para esta documentação.

3. Referência visual e princípio mecânico



Figura 2 - Conceito visual de referência da mão robótica preta
Fonte/crédito: Render conceitual gerado para o projeto SmartHand AI.

A referência visual estabelece cinco decisões de projeto: dedos segmentados, cabos/tendões aparentes ou semi-embutidos, compartimento de servos no punho, baia de eletrônica acessível e tampas removíveis para manutenção.

- Corpo principal em preto fosco ou grafite, adequado para PLA, PETG ou material equivalente de prototipagem.
- Falanges modulares para permitir substituição de uma única parte em caso de quebra.
- Pinos/parafusos nas articulações em vez de dobradiças impressas monolíticas.
- Tendões de nylon/fio de pesca passando por guias internos ou canais superficiais.
- Servomotores concentrados no punho/antebraço para reduzir massa nos dedos.
- Tampa traseira para acesso rápido a ESP32, PCA9685, conectores e distribuição de energia.

4. Problema, objetivo e público-alvo

Problema de engenharia: criar uma mão robótica de baixo custo e modular que reproduza movimentos dos dedos de forma controlada e que possa evoluir para interpretar sinais do usuário.

Objetivo geral: desenvolver e validar um protótipo funcional que integre impressão 3D, acionamento por tendões, sistemas embarcados, visão computacional e, em uma fase posterior, EMG e inteligência artificial.

4.1 Público-alvo do conceito

- Beneficiário final em potencial: pessoas com perda ou ausência de membro superior, dentro de um contexto futuro de tecnologia assistiva devidamente validada.
- Público acadêmico: estudantes e pesquisadores de Ciência da Computação, robótica, sistemas embarcados, IA e interação humano-computador.
- Profissionais e laboratórios de reabilitação: como parceiros de pesquisa, validação ergonômica e definição de requisitos, não como usuários de um dispositivo clínico pronto.
- Projetos de acessibilidade e tecnologia assistiva de baixo custo.

5. Escopo por fases

Fase	Entrada	Saída	Objetivo
MVP 1	Webcam	5 movimentos de dedos	Validar mecânica, tendões, servos e comunicação
MVP 2	Webcam + FSR	Movimento + limite de pressão	Adicionar feedback de contato
MVP 3	EMG	Abrir/fechar ou poucos gestos	Validar controle muscular
Fase IA	Múltiplos canais EMG	Classificação de gestos	Personalização por usuário
Fase futura	EMG + FSR + app	Controle adaptativo	Plataforma assistiva experimental completa

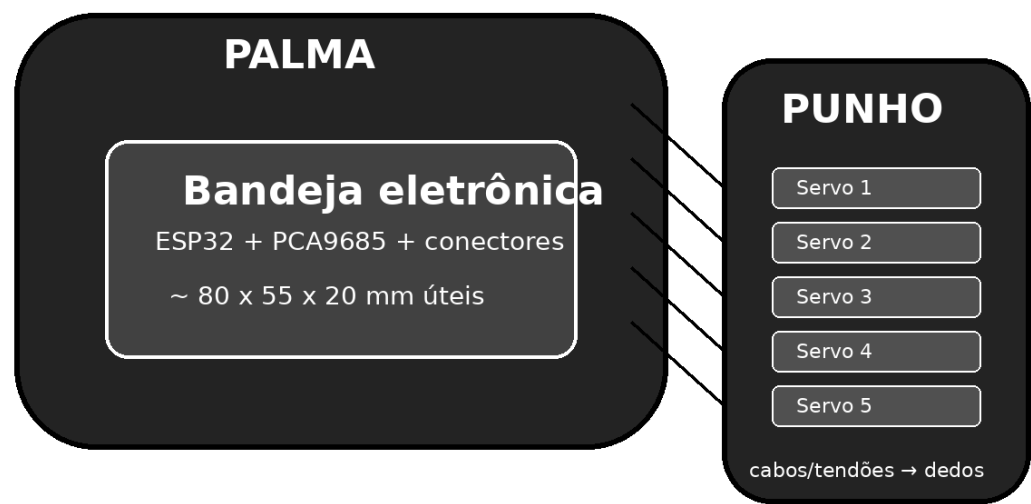
6. Projeto mecânico e impressão 3D

6.1 Arquitetura mecânica recomendada

A solução mais simples para o primeiro protótipo é usar um servo por dedo. O servo enrola ou puxa um cabo ligado a falange distal. Quando o cabo é tensionado, o dedo flexiona; a abertura pode ser feita por elástico, mola leve ou segundo cabo antagonista.

Envelope interno sugerido - conceito CAD

Dimensões abaixo são ponto de partida e devem ser ajustadas aos componentes reais.



- Tampa traseira removível com parafusos M2/M3.
- Guias de PTFE/olhais para reduzir atrito dos tendões.
- Servos preferencialmente no punho/antebraço, não nas falanges.
- Canal separado para potência dos servos e sinais.
- Prever alívio de tração e acesso ao cabo USB.

Figura 3 - Envelope interno sugerido para o primeiro CAD
Fonte/crédito: Conceito dimensional inicial; ajustar após medir hardware.

6.2 Dimensões iniciais de CAD

Elemento	Envelope inicial sugerido	Observação
Comprimento total da mão	185 a 205 mm	Ajustar ao objetivo estético e ergonômico do protótipo
Largura da palma	80 a 95 mm	Deve comportar canais de tendões e tampa
Espessura da palma	24 a 32 mm	Aumentar se a eletrônica ficar na palma
Baía útil de eletrônica	aprox. 80 x 55 x 20 mm	Ponto de partida; medir as placas escolhidas
Módulo punho/servos	aprox. 70 x 60 x 45 mm ou maior	Depende da orientação dos cinco micros servos
Folga entre peças moveis	0,3 a 0,6 mm	Faixa inicial para FDM; calibrar na impressora
Furos para eixos	dimensionar após escolher pino/parafuso	Evitar depender de pinos impressos finos

Recomendação de CAD: Modele primeiro os componentes reais como blocos simplificados com dimensões medidas no paquímetro. So depois desenhe a carenagem externa. Isso evita produzir uma mão visualmente bonita em que as placas, fios ou conectores não cabem.

6.3 Sistema de tendões

- Um cabo de flexão por dedo no MVP.
- Canal ou tubo-guia de baixo atrito atravessando a palma.
- Ponto de ancoragem na falange distal ou intermediária.
- Tambor pequeno ou braco do servo para transformar rotação em tração.

- Elástico/mola de retorno dimensionado para reabrir o dedo sem exigir força excessiva.
- Ajustador de tensão mecânico no punho para compensar alongamento do fio.
- Bordas arredondadas nos canais para evitar corte do nylon.

6.4 Materiais de impressão

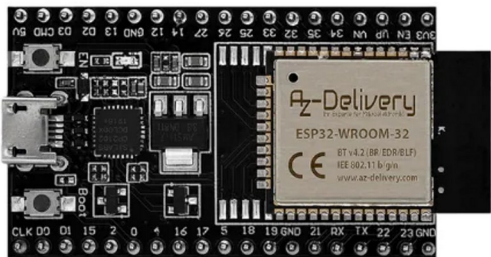
Material	Uso sugerido	Vantagem	Atenção
PLA	Primeiro protótipo e testes dimensionais	Fácil de imprimir e rígido	Menor resistência térmica
PETG	Versão mecânica mais resistente	Boa tenacidade	Pode formar fios e exigir calibração
TPU	Pontas de dedos/areas de contato	Maior aderência e flexibilidade	Não usar como estrutura principal
Nylon	Versão avançada	Boa resistência mecânica	Impressão mais exigente

7. Hardware principal

Componente	Qtd.	Papel no projeto	Fase
ESP32 DevKitC ou equivalente	1	Microcontrolador e comunicação	MVP
PCA9685 16 canais PWM	1	Geração de PWM para os servos	MVP
Microservo metal gear MG90D/semelhante	5	Acionamento dos cinco dedos	MVP
Webcam	1	Captura da mão de referência	MVP
Fonte externa regulada para servos	1	Energia dos atuadores	MVP
FSR 402 ou sensor de força equivalente	1 a 5	Feedback de pressão	MVP 2
MyoWare 2.0 ou EMG equivalente	1+	Atividade muscular	Fase EMG
Fios jumper, conectores, chave e fusível	vários	Interconexão e segurança	MVP
Nylon/fio de pesca e elasticos	vários	Tendões e retorno dos dedos	MVP
Parafusos/pinos M2-M3	vários	Eixos e montagem	MVP

8. Componentes detalhados com referência visual

8.1 ESP32 DevKitC



O ESP32 funciona como controlador embarcado. Recebe os cinco comandos de flexão pela porta serial, aplica limites e envia as posições ao PCA9685 via I2C.

- Placa compacta de prototipagem com GPIOs acessíveis.
- A família ESP32 integra conectividade Wi-Fi e Bluetooth em diversas variantes.
- No ESP32 clássico, o projeto pode iniciar com SDA em GPIO21 e SCL em GPIO22.
- Não usar o pino de alimentação do ESP32 para fornecer corrente aos cinco servos.

Referência fotográfica do componente

Fonte/crédito: Foto recortada de kit AZ-Delivery; especificações conferidas na documentação Espressif.

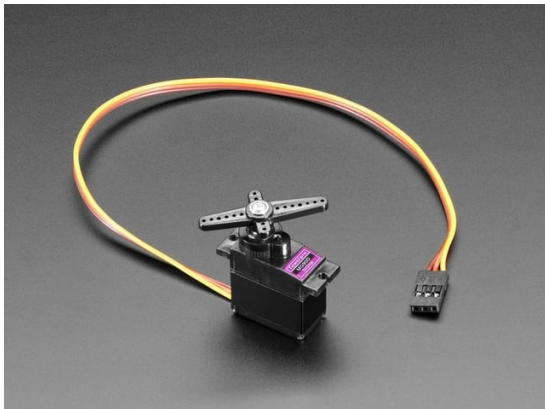
8.2 PCA9685 - controlador PWM

O PCA9685 centraliza o controle dos servomotores. Ele recebe comandos por I2C e gera sinais PWM independentes, evitando ocupar vários canais de PWM do microcontrolador.

- 16 canais de saída PWM.
- Resolução de 12 bits.
- Comunicação I2C.
- Adequado para controlar mais servos do que o necessário no MVP, deixando margem para evolucoes.

Referência fotográfica do componente

Fonte/crédito: Foto recortada de kit AZ-Delivery; arquitetura validada com documentação Adafruit/NXP.

8.3 Microservo MG90D / servo metal gear equivalente

Um microservo por dedo simplifica a primeira versão. A versão MG90D é uma referência compacta com engrenagens metálicas; modelos equivalentes podem ser usados após verificação de dimensões e torque.

- Dimensões publicadas do MG90D: aproximadamente 22,8 x 12,2 x 28,5 mm.
- Faixa de alimentação publicada: 4,8 a 6 V DC.
- Calibrar pulsos e curso antes de conectar o tendão.
- O torque necessário depende do raio do tambor, atrito, geometria do dedo e objeto manipulado.

Referência fotográfica do componente

Fonte/crédito: Adafruit, produto MG90D High Torque Metal Gear.

8.4 FSR 402 - sensor de força



O FSR pode ser colocado na ponta do dedo ou em uma superfície de contato. Sua resistência diminui quando a força aplicada aumenta, permitindo estimar pressão relativa por meio de um divisor resistivo ligado ao ADC.

- Área ativa do modelo 402: aproximadamente 12,7 mm de diâmetro.
- Sensor fino e simples de integrar em pontas de dedos.
- Não substitui uma célula de carga quando é necessária medição de força de alta precisão.
- Para o MVP, use primeiro como limite relativo de segurança, após calibração experimental.

Referência fotográfica do componente

Fonte/crédito: Foto de produto: Visha World. Especificações: Interlink Electronics.

8.5 MyoWare 2.0 - sensor EMG (fase futura)



O MyoWare converte a atividade eletromiográfica superficial em sinais que podem ser lidos por microcontroladores. No SmartHand, ele é reservado para uma fase posterior, depois que a mecânica estiver validada.

- Pode fornecer modos de saída como EMG bruto, retificado e envelope, dependendo da configuração.
- A documentação atual do produto destaca compatibilidade típica com 3,3 V a 5 V.
- Um único canal é mais adequado para detecção simples de contração; vários gestos podem exigir múltiplos canais e treinamento.
- O próprio fabricante informa que o sensor não é destinado a diagnosticar, tratar ou prevenir doenças.

Referência fotográfica do componente

Fonte/crédito: SparkFun / Advancer Technologies, MyoWare 2 Muscle Sensor.

9. Arquitetura elétrica e ligações

9.1 Ligação ESP32 - PCA9685

Sinal	ESP32	PCA9685	Observação
Lógica	3,3 V / conforme breakout	VCC	Alimentação da lógica, não dos servos
Terra	GND	GND	Terra comum com a fonte dos servos
Dados I2C	GPIO21 (base)	SDA	Pode ser remapeado

Sinal	ESP32	PCA9685	Observação
Clock I2C	GPIO22 (base)	SCL	Pode ser remapeado
Potência servo	Fonte externa	V+	Dimensionar pela corrente dos atuadores

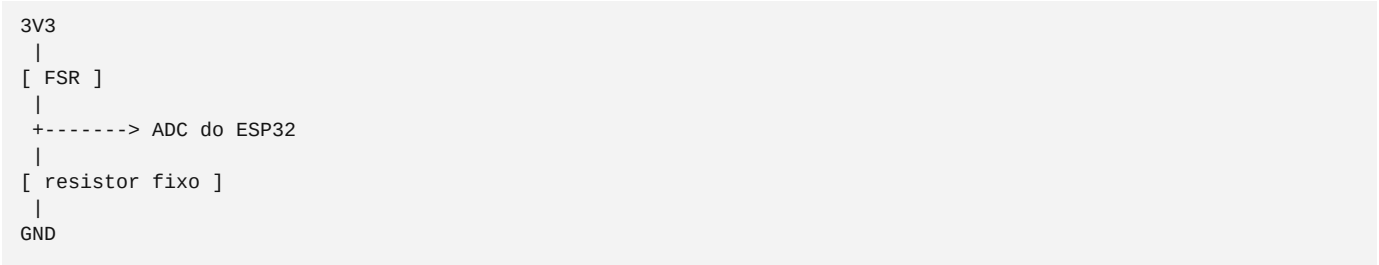
9.2 Canais dos dedos

Canal PCA9685	Dedo	Convenção lógica
0	Polegar	0 aberto / 180 fechado
1	Indicador	0 aberto / 180 fechado
2	Médio	0 aberto / 180 fechado
3	Anelar	0 aberto / 180 fechado
4	Mínimo	0 aberto / 180 fechado

Alimentação: O PCA9685 possui alimentação de lógica separada da alimentação V+ usada pelos servos. Os cinco servos devem receber energia de fonte externa regulada, com terra comum ao ESP32. A fonte deve ser dimensionada a partir de medição real de corrente e do pior caso de movimentos simultâneos.

9.3 FSR em divisor resistivo

Um FSR não deve ser ligado ao ADC como se fosse uma fonte de tensão. Use um divisor resistivo e leia a tensão resultante. O valor de resistor fixo e o limiar devem ser definidos experimentalmente conforme a faixa de pressão do protótipo.



10. Arquitetura de software

Módulo	Tecnologia	Responsabilidade
Captura	OpenCV	Abrir webcam e fornecer frames
Rastreamento	MediaPipe Hand Landmarker	Detectar landmarks da mão
Conversão	Python + NumPy	Transformar ângulos dos dedos em 0-180
Comunicação	PySerial	Enviar cinco valores ao ESP32
Controle embarcado	Arduino C++ / ESP32	Validar limites, emergência e comandos
Atuação	Adafruit PWM Servo Driver	Gerar pulsos por PCA9685
EMG futuro	ESP32 ADC + Python	Coletar dados e treinar classificador

10.1 Dependências do MVP Python

```
opencv-python>=4.10
mediapipe>=0.10
pyserial>=3.5
numpy>=1.26
pandas>=2.2
scikit-learn>=1.5
joblib>=1.4
```

O MediaPipe Hand Landmarker atual usa um arquivo de modelo .task. O caminho para esse arquivo é informado por parâmetro ao programa. O objetivo do código-base é facilitar a validação; limiares e mapeamentos devem ser calibrados para a mão de referência e para o mecanismo impresso.

11. Código-base

Pacote completo: A documentação inclui trechos centrais. O arquivo SmartHandAI_CodeBase.zip entregue junto com este documento contém os arquivos completos e separados por pasta.

11.1 Python - rastreamento da mão e envio Serial

```
FINGER_TRIPLETS = [
    (2, 3, 4),    # polegar: MCP, IP, TIP
    (5, 6, 8),    # indicador: MCP, PIP, TIP
    (9, 10, 12),  # médio
    (13, 14, 16), # anelar
    (17, 18, 20), # mínimo
]

# Ângulo articular aproximado. Ajuste na calibração.
STRAIGHT_DEG = 175.0
BENT_DEG = 60.0
SMOOTHING = 0.35

def joint_angle(a, b, c) -> float:
    """Retorna o ângulo ABC, em graus, usando coordenadas 3D normalizadas."""
    va = np.array([a.x - b.x, a.y - b.y, a.z - b.z], dtype=float)
    vc = np.array([c.x - b.x, c.y - b.y, c.z - b.z], dtype=float)

    denom = np.linalg.norm(va) * np.linalg.norm(vc)
    if denom < 1e-8:
        return 180.0

    cosang = float(np.clip(np.dot(va, vc) / denom, -1.0, 1.0))
    return math.degrees(math.acos(cosang))

def angle_to_servo(finger_angle: float) -> int:
    """
    Converte:
    ~175° (dedo estendido) -> 0° de fechamento do servo
    ~60° (dedo flexionado) -> 180° de fechamento do servo
    """
    value = np.interp(
        finger_angle,
        [BENT_DEG, STRAIGHT_DEG],
        [180.0, 0.0],
    )
    return int(np.clip(value, 0, 180))

def finger_commands(landmarks) -> List[int]:
    commands = []
```

```

for a_idx, b_idx, c_idx in FINGER_TRIPLETS:
    angle = joint_angle(
        landmarks[a_idx],
        landmarks[b_idx],
        landmarks[c_idx],
    )
    commands.append(angle_to_servo(angle))
return commands

def draw_landmarks(frame, landmarks) -> None:
    """Desenha os 21 landmarks sem depender do drawing_utils legado."""
    h, w = frame.shape[:2]
    points = []
    for lm in landmarks:
        x, y = int(lm.x * w), int(lm.y * h)
        points.append((x, y))
        cv2.circle(frame, (x, y), 4, (255, 255, 255), -1)

    connections = [
        (0,1),(1,2),(2,3),(3,4),
        (0,5),(5,6),(6,7),(7,8),
        (5,9),(9,10),(10,11),(11,12),
        (9,13),(13,14),(14,15),(15,16),
        (13,17),(17,18),(18,19),(19,20),
        (0,17),
    ]
    for i, j in connections:
        cv2.line(frame, points[i], points[j], (190, 190, 190), 2)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--port", required=True, help="Porta serial do ESP32")
    parser.add_argument("--baud", type=int, default=115200)
    parser.add_argument("--model", required=True, help="Caminho para hand_landmarker.task")
    parser.add_argument("--camera", type=int, default=0)
    args = parser.parse_args()

    ser = serial.Serial(args.port, args.baud, timeout=0.05)
    time.sleep(2.0)

    cap = cv2.VideoCapture(args.camera)
    if not cap.isOpened():
        raise RuntimeError("Não foi possível abrir a câmera.")

    base_options = python.BaseOptions(model_asset_path=args.model)
    options = vision.HandLandmarkerOptions(
        base_options=base_options,
    )
    # ... trecho reduzido na documentação; código completo no ZIP.

```

11.2 ESP32 + PCA9685 - controle dos cinco dedos

```

#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40);

constexpr uint8_t NUM_FINGERS = 5;
constexpr uint8_t SERVO_CH[NUM_FINGERS] = {0, 1, 2, 3, 4};

// Pulsos conservadores a 50 Hz. Calibre por servo/mecânica.
constexpr uint16_t PULSE_MIN[NUM_FINGERS] = {155, 155, 155, 155, 155};
constexpr uint16_t PULSE_MAX[NUM_FINGERS] = {455, 455, 455, 455, 455};

```

```

// Se um servo fechar no sentido oposto, troque false -> true.
constexpr bool INVERT[NUM_FINGERS] = {false, false, false, false, false};

constexpr int EMERGENCY_PIN = 27;

// FSRs opcionais. GPIOs de ADC adequados variam por placa.
constexpr bool USE_FSR = false;
constexpr int FSR_PIN[NUM_FINGERS] = {34, 35, 32, 33, 36};
constexpr int FSR_LIMIT[NUM_FINGERS] = {2800, 2800, 2800, 2800, 2800};

int currentClose[NUM_FINGERS] = {0, 0, 0, 0, 0};
String inputLine;

uint16_t closeToPulse(uint8_t finger, int closeAmount) {
    closeAmount = constrain(closeAmount, 0, 180);

    if (INVERT[finger]) {
        closeAmount = 180 - closeAmount;
    }

    return map(
        closeAmount,
        0, 180,
        PULSE_MIN[finger],
        PULSE_MAX[finger]
    );
}

void setFinger(uint8_t finger, int closeAmount) {
    closeAmount = constrain(closeAmount, 0, 180);

    // Se houver FSR e a força já estiver alta, bloqueia fechamento adicional.
    if (USE_FSR) {
        int force = analogRead(FSR_PIN[finger]);
        if (force > FSR_LIMIT[finger] && closeAmount > currentClose[finger]) {
            closeAmount = currentClose[finger];
        }
    }

    currentClose[finger] = closeAmount;
    pwm.setPWM(
        SERVO_CH[finger],
        0,
        closeToPulse(finger, closeAmount)
    );
}

void openHand() {
    for (uint8_t i = 0; i < NUM_FINGERS; i++) {
        setFinger(i, 0);
    }
}

void disableServos() {
    // 4096 é o bit "full off" do PCA9685.
    for (uint8_t i = 0; i < NUM_FINGERS; i++) {
        pwm.setPWM(SERVO_CH[i], 0, 4096);
    }
}

bool parseFiveIntegers(const String& line, int out[NUM_FINGERS]) {
    int start = 0;

    for (uint8_t i = 0; i < NUM_FINGERS; i++) {
        int comma = line.indexOf(',', start);

        String token;
        if (i < NUM_FINGERS - 1) {
            if (comma < 0) return false;
            token = line.substring(start, comma);
            start = comma + 1;
        }
    }
}

```

```

    } else {
        token = line.substring(start);
    }

    token.trim();
    if (token.length() == 0) return false;

    out[i] = constrain(token.toInt(), 0, 180);
}

return true;
}

void setup() {
    # ... trecho reduzido na documentação; código completo no ZIP.

```

11.3 Leitura de EMG - base para fase futura

```

/*
    SmartHand AI - Leitura base de EMG (envelope)
    -----
    Exemplo para MyoWare 2.0 usando saída ENV/Envelope.

    ATENÇÃO:
    - Projeto acadêmico, não dispositivo médico.
    - Siga a documentação do fabricante para eletrodos, alimentação e segurança.
    - Não utilize este código como sistema clínico.
*/

constexpr int EMG_PIN = 34;
constexpr uint32_t SAMPLE_HZ = 200;
constexpr uint32_t PERIOD_US = 1000000UL / SAMPLE_HZ;

uint32_t nextSample = 0;

void setup() {
    Serial.begin(115200);
    analogReadResolution(12);
    nextSample = micros();
}

void loop() {
    uint32_t now = micros();

    if ((int32_t)(now - nextSample) >= 0) {
        nextSample += PERIOD_US;

        int emg = analogRead(EMG_PIN);

        // formato: tempo_ms,emg
        Serial.print(millis());
        Serial.print(',');
        Serial.println(emg);
    }
}

```

11.4 Coleta e treinamento de um baseline EMG

A pasta Python inclui um coletor de CSV e um classificador Random Forest de referência. O baseline extrai características simples de janelas do sinal, como media, desvio padrao, RMS, mínimo, maximo e amplitude pico-a-pico.

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

```

```

def rms(x):
    return float(np.sqrt(np.mean(np.square(x))))

def make_windows(df, window=40, step=20):
    X, y = [], []

    for label, group in df.groupby("label"):
        signal = group["emg"].to_numpy(dtype=float)

        for start in range(0, len(signal) - window + 1, step):
            x = signal[start:start + window]
            X.append([
                float(np.mean(x)),
                float(np.std(x)),
                rms(x),
                float(np.min(x)),
                float(np.max(x)),
                float(np.ptp(x)),
            ])
            y.append(label)

    return np.asarray(X), np.asarray(y)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--csv", default="emg_dataset.csv")
    ap.add_argument("--model", default="emg_model.joblib")
    args = ap.parse_args()

    df = pd.read_csv(args.csv)
    X, y = make_windows(df)

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.25,
        random_state=42,
        stratify=y,
    )

    clf = RandomForestClassifier(
        n_estimators=200,
        random_state=42,
        class_weight="balanced",
    )
    clf.fit(X_train, y_train)

    pred = clf.predict(X_test)
    print(classification_report(y_test, pred))

    joblib.dump(clf, args.model)
    print(f"Modelo salvo em: {args.model}")

if __name__ == "__main__":
    main()

```

Limite do baseline: Um único sensor de envelope EMG é adequado para demonstrar repouso versus contração. Reconhecer vários gestos de mão de forma robusta normalmente exige melhor posicionamento, mais de um canal, coleta de dados por usuário e validação do modelo.

12. Preparação do ambiente de desenvolvimento

12.1 Computador

1. Instalar Python 3.10 ou superior.
2. Criar um ambiente virtual.
3. Instalar as dependências do arquivo requirements.txt.
4. Baixar o modelo Hand Landmarker compatível com o MediaPipe Tasks e informar seu caminho no parâmetro --model.
5. Conectar a webcam e testar o OpenCV isoladamente.
6. Conectar o ESP32 por USB e identificar a porta serial.

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# Linux/macOS
source .venv/bin/activate

pip install -r python/requirements.txt

python python/hand_tracker_serial.py --port COM5 --model hand_landmarker.task
```

12.2 Arduino IDE / firmware

7. Instalar o suporte da placa ESP32 no Arduino IDE.
8. Instalar a biblioteca Adafruit PWM Servo Driver e suas dependências.
9. Carregar primeiro servo_calibration.ino.
10. Calibrar um servo sem tendão e sem carga.
11. Somente depois carregar smarthand_pca9685.ino.

13. Procedimento de calibração

13.1 Calibração mecânica de cada dedo

12. Desconectar o tendão do dedo.
13. Executar o firmware de calibração.
14. Mover o servo lentamente para encontrar uma posição segura de abertura.
15. Registrar o pulso correspondente.
16. Mover lentamente ate uma posição segura de fechamento sem forçar o fim de curso.
17. Registrar o pulso correspondente.
18. Montar o tendão com pequena pre-tensão.
19. Repetir o teste e reduzir o curso caso exista travamento.
20. Atualizar PULSE_MIN, PULSE_MAX e INVERT no firmware principal.

13.2 Calibração do FSR

21. Ler o valor sem contato.
22. Pressionar levemente o sensor e registrar o valor.
23. Repetir com pressão moderada e com o objeto de teste.
24. Definir um limiar de bloqueio abaixo da faixa que deforme ou danifique o objeto.
25. Validar o limiar em varias repetições.

13.3 Calibração de visão computacional

- Ajustar BENT_DEG e STRAIGHT_DEG no Python.

- Validar o polegar separadamente, pois sua orientação difere dos demais dedos.
- Aumentar suavização se os servos apresentarem tremor.
- Reduzir a taxa de comandos se a comunicação serial estiver saturando.

14. Plano de montagem

Etapa	Entrega	Critério de aceite
1. CAD de um dedo	Falanges + articulações	Move manualmente sem travar
2. Tendão de um dedo	1 servo + 1 dedo	Abre e fecha 50 vezes
3. Mão completa	5 dedos montados	Todos articulam manualmente
4. Punho de servos	5 servos instalados	Sem colisão e com acesso aos cabos
5. Eletrônica	ESP32 + PCA9685	Comunicação I2C funcional
6. Firmware	Comandos manuais	Cada dedo responde individualmente
7. Visão	MediaPipe + Serial	Mão virtual gera 5 comandos
8. Integração	Webcam -> mão 3D	Movimentos básicos reproduzidos
9. FSR	1 ou mais sensores	Fechamento adicional e bloqueado no limiar
10. EMG futuro	Sinal muscular	Repouso/contração separados

15. Plano de testes e metricas

Teste	Métrica	Método	Meta inicial de protótipo
Repetibilidade	taxa de ciclos sem falha	100 ciclos por dedo	>= 90% sem travamento
Latência	ms gesto -> inicio movimento	timestamp no PC + vídeo	registrar e comparar iteracoes
Precisão de movimento	erro de estado/ângulo	comparar comando x resposta visual	suficiente para demonstração
Pegada	taxa de sucesso	10 tentativas por objeto	documentar por objeto
FSR	bloqueio por pressão	aplicar contato progressivo	sem sobrecurso após limiar
Temperatura	aquecimento de servo/placa	inspeção periódica	sem aquecimento anormal
Consumo	corrente de pico	fonte com medição/multímetro	dimensionar alimentação real

15.1 Objetos de teste

- Bola de espuma: grande tolerância a pressão.
- Copo plástico vazio: avalia estabilidade lateral.
- Garrafa pequena: exige pegada de potência.
- Caneta grossa: exige precisão e fechamento parcial.
- Bloco de espuma ou objeto frágil substituto: demonstra limite de pressão sem risco.

16. Riscos e segurança

Risco	Causa	Mitigação
Beliscamento	articulações moveis	manter dedos humanos afastados; testar em baixa velocidade
Servo travado	curso excessivo/tendão curto	calibração sem carga e limites de pulso
Queda de tensão	vários servos simultâneos	fonte externa adequada e terra comum
Aquecimento	servo em stall	desligar ao detectar travamento; limitar força

Risco	Causa	Mitigação
Cabo rompido	atrito em arestas	guias arredondadas e inspeção periódica
Comando instável	tracking ruidoso	suavização, zona morta e watchdog
Contato EMG	uso inadequado	seguir fabricante; protótipo não clínico; supervisão adequada

EMG: O sensor MyoWare é apresentado pelo fabricante como ferramenta para projetos e pesquisa, não como equipamento para diagnosticar ou tratar condicoes de saude. A fase EMG deve seguir rigorosamente a documentação do fabricante e protocolos de segurança da instituição.

17. Estrutura de arquivos entregue

```
SmartHandAI_CodeBase/  
|-- README.md  
|-- docs/  
|   |-- wiring.md  
|-- esp32/  
|   |-- smarthand_pca9685.ino  
|   |-- servo_calibration.ino  
|   |-- emg_stream.ino  
|-- python/  
|   |-- hand_tracker_serial.py  
|   |-- collect_emg_dataset.py  
|   |-- train_emg_baseline.py  
|   |-- requirements.txt
```

18. Roadmap recomendado

Sprint	Foco	Resultado
1	CAD de um único dedo	Geometria articulada validada
2	Servo + tendão	Primeiro dedo automatizado
3	Palma e cinco dedos	Mão mecânica completa
4	Punho e eletrônica	ESP32/PCA9685 alojados
5	Firmware	Controle individual dos cinco servos
6	Visão computacional	Mão real réplica movimentos básicos
7	FSR	Feedback de contato em pelo menos um dedo
8	Refino	Tampa, cabos, acabamento preto
9	EMG experimental	Abrir/fechar por atividade muscular
10	IA	Dataset e classificação personalizada

19. Roteiro de demonstração do protótipo

- 26. Exibir a imagem conceitual preta e apresentar a pergunta: 'E se a mão pudesse aprender como o usuário quer se movimentar?'
- 27. Mostrar o protótipo aberto e apontar rapidamente falanges, tendões, punho e tampa de manutenção.
- 28. Exibir em tela a webcam com os landmarks do MediaPipe.
- 29. Abrir e fechar a mão de referência e demonstrar a réplica na mão robótica.

30. Executar uma pinça e uma pegada simples.
31. Mostrar o dado de pressão do FSR, se essa fase estiver pronta.
32. Encerrar explicando a evolução para EMG e IA como próxima etapa, sem afirmar que o protótipo já é uma prótese clínica.

20. Checklist antes de imprimir a versão completa

- Todos os servos foram medidos com paquímetro.
- ESP32 e PCA9685 cabem com conectores e cabos instalados.
- Existe acesso ao USB sem desmontar a mão inteira.
- A tampa pode ser removida sem desconectar os tendões.
- Cada cabo possui caminho sem arestas vivas.
- Ha espaço para ajustar a tensão dos tendões.
- Os parafusos das articulações não colidem com os cabos.
- A palma possui bosses/porcas ou insertos para parafusos.
- A entrada da fonte possui alívio de tração.
- Existe chave geral e ponto fácil de desligamento.
- Os dedos possuem batentes mecanicos de curso.
- As peças críticas podem ser reimpressas separadamente.

21. Referencias tecnicas e créditos de imagens

Vídeo de inspiração F.L.E.X. - My Most Advanced Robotic Hand Yet. YouTube, ID NimRONJeQDY.

Google AI Edge - MediaPipe Hand Landmarker. Documentação oficial, acesso em agosto de 2026.

OpenCV - VideoCapture / Vídeo I/O. Documentação oficial OpenCV.

Espressif - ESP32-DevKitC. Página e guia oficiais da placa.

Adafruit - PCA9685 16-Channel PWM/Servo Driver. Especificações e orientações de ligação.

Adafruit - Micro Servo MG90D High Torque Metal Gear. Dimensões, tensão e características do servo de referência.

SparkFun / Advancer Technologies - MyoWare 2 Muscle Sensor. Especificações e aviso de uso não médico.

Interlink Electronics - FSR Model 402. Especificações do sensor de força.

AZ-Delivery - ESP32 Servo Control Kit. Imagem utilizada para ilustrar ESP32, SG90 e PCA9685.

Visha World - FSR 402 Force Sensor. Imagem fotográfica do FSR utilizada apenas como referência visual.

22. Conclusao

A melhor estratégia para o SmartHand AI é reduzir o risco técnico em etapas: primeiro provar um dedo, depois cinco dedos, depois integrar a visão computacional e apenas entao adicionar feedback de força e EMG. Essa sequência transforma a referência visual em um projeto reproduzível e mensurável, com componentes acessíveis, código modular e espaço físico planejado para manutenção.

O primeiro objetivo de sucesso não deve ser construir uma prótese clínica completa, mas demonstrar uma plataforma robótica de tecnologia assistiva capaz de reproduzir movimentos, medir resultados e evoluir para controle muscular inteligente.

Próxima etapa de engenharia: Modelar em CAD somente um dedo + módulo de servo e imprimir essa unidade. Quando o mecanismo completar pelo menos 50 a 100 ciclos sem travar, replicar a geometria para os demais dedos e desenhar a palma definitiva.